

COMP-3150 FINAL REVIEW CLASS: LECTURE

NOTES (LAST CLASS) Covered on Mar. 31, 2026

COMP 3150 WINTER 2026 FINAL REVIEW CLASS:

This class has two parts:

1. Final Examination Reviews: A review and summary of all materials covered in this course in Winter 2026 term. The schedule of final examination, final exam viewing and final exam make-up are also provided.
2. Final Exam Practice Discussions: Some solutions to the final examination practice given to class earlier on brightspace for Winter 2026. This is in honor of those students who have attempted to solve all or some of these questions in preparation for the final exams.

Good Luck in you Finals !!

Part I: The Final Exam Reviews

Comp-3150-01 (60-315-01)

Winter 2026 Final Exam Preparation

Upcoming Class Events

Final Exam Date (covers materials from chs. 1, 2, 3, 5, 14, 6, 7, 8, 17 of book), lecture slides and written lecture notes and examples in class.	Date: Mon. Apr. 20, 2026. Time: 3:30pm to 6.00pm Where: in person in EDUC 1101
Tentative Exam Viewing Date	Thurs., Apr. 23, 2026 (10:00am to 4:00pm) in LT 8118, meeting with supervising GA/TA in a classroom.

*** GOOD LUCK ****

NOTE: In the event you cannot write the exam due to illness or some other emergency (approved CS medical certificate required for proof), call/send email to the CS office (csinfo@uwindsor.ca) or send email to me within one day or as soon as possible to inform. There is only one make-up exam for all, who could not write final and are pre-approved by me to write the make-up exam; that is on Friday, May 8, 2026 (1:00 to 3:30 pm) in LT 8118.

We hope to mount exam viewing sessions on **Thus. Apr. 23, 2026 (10:00am to 4:00pm)** in LT 8118, meeting with supervising GA/TA in a classroom, **subject to we, completing the marking. Viewing is as scheduled unless otherwise announced.**

Student Name and id: _____

These questions in the final exam practice are to help with preparing for final exams. To prepare well, read through all of the lecture slides and the notes on both brightspace and that you have been writing to ensure you feel comfortable with the materials taught in the course. Refer to the text book when you want more information about some concepts in the lecture slides.

Format of the final Exam: 2 hours 30 minutes with two Sections (A and B).

- Section A has 15 multiple choice questions for a total of 30 marks while Section B has 4 full questions worth a total of 70 marks.

- Check the final exam practice questions for help with preparation. Note that no solutions are provided online for the final exam practice questions. The only review to these questions are in class with students during the lecture.

We have covered **materials from Chs. 1, 2, 3, 5, 14, 6, 7, 8, and 17 of the course book** and should know: Chs 1 – 3: DBMS concepts and architecture, data models based on entities and relationships, data base design using the Entity-Relationship model (ER model), page 83 of book and slide 19 of Chapter 3 notes provide a summary of notations used for representing a database in ER diagrams. We should be able to design a database using ER model, convert between ER models and relational model and vice versa.

Ch. 5: Relational data model concepts, relational database design and constraints. Some concepts are: Database schemas, database states, primary key, candidate key, superkey, foreign key, domains, attributes, relations, degree of a relation, cardinality of a relation, system catalog, Null values. Some constraints are domain, key, foreign key, referential integrity, and entity integrity. Relational model operations are those for creating and updating the schema and those for creating and updating data and retrieving data. Those operations for data manipulation that may violate database constraints are Insert operation, delete operation, and update operations.

Ch. 14: Database design guidelines and theories of normal forms based on functional dependencies. For good database design, attributes from multiple entity types (e.g. student, course) should not be combined in one entity. Such design would cause base relations to not be in at least 3NF and suffer from anomalies due to redundant storage of data. Design should minimize the need for attributes that have a lot of NULL values. To minimize generation of spurious tuples (not in the original relations) from natural joins of relations, design relations that can be joined on primary key, foreign key attribute pairs. Functional dependencies (FDs) are constraints that are derived from the meaning and interrelationships of the data attributes. The FD: $X \rightarrow Y$ holds in a relation if whenever any two tuples have the same value for X, they must have the same value for Y. Normalization is the process of decomposing badly designed relations into smaller relations. Normal form uses keys and FDs to certify whether a schema is in a particular normal form such as 1NF, 2NF, 3NF and BCNF. Relations that are in 3NF and higher normal forms are considered to be normalized. 1NF allows relations to store only single values for each attribute of a tuple. A relation R is in 2NF if every non-key attribute is fully functionally dependent on the key (that is, the only determinants (X) of non-key attributes (Y) in the relation R, have no subsets of them (X) also determining any non-key attributes (Y)). A relation is in 3NF if it is in 2NF and no non-key attribute of R is transitively dependent on the primary key (that is, the only determinants (X) of non-key attributes (Y) in the relation R, have no other non-key attribute (Z) also determining any non-key attributes (Y)). A relation is in BCNF if whenever an FD $X \rightarrow Y$ holds in R, then X is a superkey of R (that is, the superkey (X) is the only determinant and no non-key attribute should determine any part of the superkey). While 3NF allows R that are in either the FD condition (a): $X \rightarrow A$ where X is a superkey or (b): A is a key attribute of R (that is, a non-key attribute X can determine part of a key); the BCNF does not allow condition (b). Fig. 14.12 on page 485 of book steps you through the LOTS relation in 1NF, 2NF, 3NF, while Fig 14.13 shows you its decomposition into BCNF.

Ch. 6: Basic SQL: for data definition language (DDL) and data manipulation language (DML). SQL DDL are for creating database schemas, database tables, modifying table schemas, deleting table schemas with constraints. SQL DML are creating, deleting and updating data and for data retrieval. The SQL DDL commands include: CREATE TABLE, DROP TABLE, ALTER TABLE commands. The SQL DML for data retrieval include the SELECT-FROM-WHERE command in spj form, nested form, with aggregate functions and GROUP BY, HAVING and ORDER BY clauses. The SQL DML for data modification include INSERT INTO, DELETE-FROM-WHERE, UPDATE-SET-WHERE statements.

- Ch. 7: More details of SQL DML for more complex data retrievals (not just the select-project-join queries) including nested queries, aggregate functions (SUM, COUNT, AVG, MIN, MAX) with group by, having and order by clauses are discussed. SQL for creating views and more discussions of schema modifications using DROP TABLE, ALTER TABLE are discussed. Slides 36 and 37 of Ch. 7 notes has a summary of SQL commands and syntax.
- Ch. 8: Relational algebra unary operations of (SELECT (σ), PROJECT (π) and RENAME(ρ)) for querying databases are discussed. The relational algebra binary operations of (JOIN ($\bowtie$) and DIVISION ($\div$)) are discussed. The set operations of UNION ($\cup$), INTERSECTION ($\cap$), DIFFERENCE (MINUS) ($-$), CARTESIAN PRODUCT (CROSS PRODUCT)($\times$) are discussed. Use of aggregate functions in relational algebra querying is discussed. The tuple relational calculus (TRC) for expressing retrieval queries in the form of $\{t \mid \text{COND}(t)\}$ was also covered. Cond(t) can be a formula consisting of an atom such as Employee(t), t.salary > 30000; or an atom connected with logical operators (such as AND, OR, NOT); and also can include existential (e.g. ($\exists d$)) and universal quantifiers (e.g., ($\forall t$)).
- Ch. 17: Use of Index files for speeding up database records from secondary disk storage is discussed. General description of index file structures as alternative access paths to primary data file on disk is given. Single level ordered indexes store the secondary index file on disk as a file ordered on the index field where each index key also contains a second field of the index file a pointer to the disk address of the index key record in the primary data file. To reduce on the number of disk accesses, multi level tree structured indexes such as ISAM, B-tree and B+-tree indexes are used. These multi level indexes are based on the binary search technique and have search time complexity of $O(\log_{\text{fanout}} bi)$ where fanout is the number of pointers that can be accommodated on each node of the tree. We focused in detail, on the construction of the B-tree and B+-tree and discussed the search, insert and delete operations on the B-tree and B+-tree. The differences between the B-tree and B+tree are discussed.

**

Part 2 of this Review Class: Final Exam Practice Discussions

The final exam practice posted to the class through brightspace is in the same format as the final exam with Section A (with 15 multiple choice questions) and Section B (with 4 questions). This year, I have reduced the mark for question 4 of Section B so that it has only 10 marks instead of previous 15 marks. Note that the final examination practice is provided so students actually tried to solve the questions and bring any problems they have for discussion. I usually do not post solution to practice tests except as a discussion in a review class. Thus, for those who solved the practice test, please, cross check your answers with the following. I will discuss only some questions in section B. Section A.

I also provide the instructions for the FINAL EXAMINATION below before discussing the Final exam practice.

**

INSTRUCTIONS (Please Read Carefully)

Examination Period is 2 hours 30 minutes

Answer all questions. Write your answers in the spaces provided in the question paper. This is closed book and closed notes test.

Total Marks =50. Total number of sections = 2

Please read questions carefully! Misinterpreting a question intentionally or unintentionally results in getting a “ZERO” for that question. Good Luck!!!

.....
CONFIDENTIALITY AGREEMENT & STATEMENT OF HONESTY

I confirm that I will keep the content of this assignment/examination confidential.

I confirm that I have not received any unauthorized assistance in preparing for or doing examination. I confirm knowing that a mark of 0 may be assigned for copied work.

For Test/Examination in Comp 3150: (additional rules to be observed):

- 1. I confirm that I agree to write this final examination as a closed book examination.**
- 2. I confirm that I am the student with the name and student id signed below.**
- 3. I confirm that I agree to not send email, chat, text or talk in any way to people other than the instructor or proctoring GA of this course during this examination.**
- 4. I confirm that I agree to not engage in copying or cheating during this examination.**

Student Signature

Student Name (please print)

Student I.D. Number

Date

For marking purposes only (This part not to be filled by students)

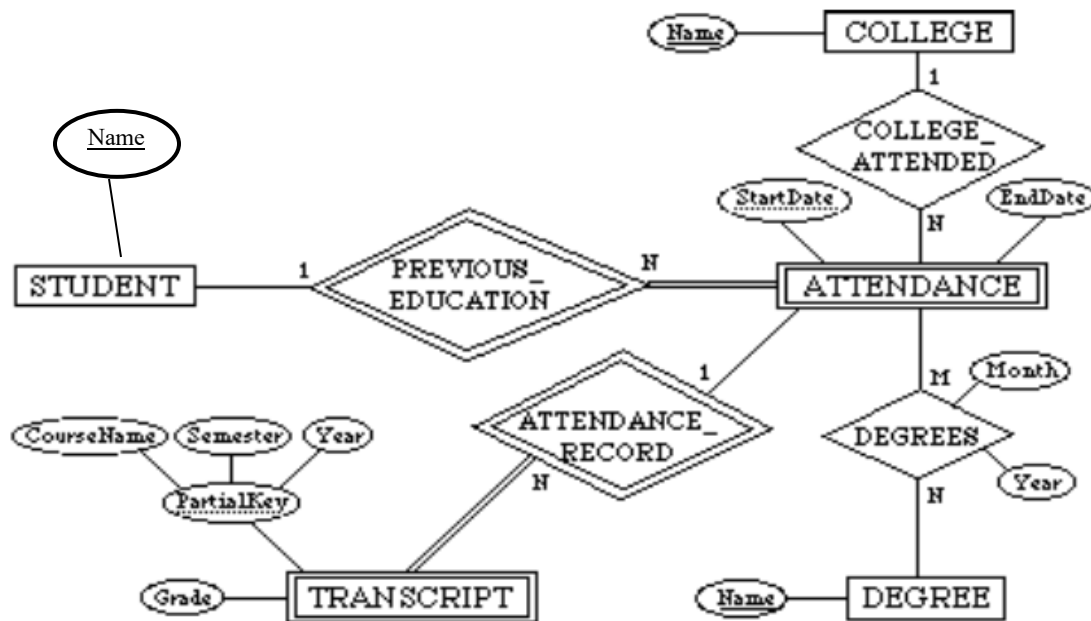
Question	Mark
Section A (30 marks for 15 multiple choice questions)	/30
Section B	
Que 1 (20 marks)	/20
Que 2 (15 marks)	/15
Que 3 (20 marks)	/20
Que 4 (15 marks)	/15
Total	/100

Solution (30 marks)

Question	Answer
1	d
2	c
3	e
4	c
5	c
6	d
7	e
8	b
9	e
10	b
11	b
12	e
13	d
14	a
15	b

Some Section B Solutions

Fig. 2: The ER model Diagram for a UNIVERSITY database Database



Consider the ER diagram of Fig 2 for a UNIVERSITY database that is used to keep track of students' transcripts. This is similar but not identical to the database shown in the Figure 1.2 in the book used in class.

Question 1: (total marks: 20 for 10 + 6 + 4)

- Define a relational database schema representation for this database system ER model, clearly defining the table schemas for 6 of the 9 tables for entities and relationships, with their attributes, identifying their primary keys and foreign key attributes constraints. Include in your 6 the two tables **College_Attended** and **Attendance_Record**.
(10 marks)
- Write all the SQL instructions to create the 6 of the database tables including the two tables **College_Attended** and **Attendance_Record** with all their primary and foreign key constraints specified.
(6 marks)
- Write an SQL query for the English query "Print the student name, the college name, the start date and end date of a student named Pete Adams".
(4 marks)

Solution for Question 1 (20 marks for all of a to c)

Sub questions	Answers
a. Relation DB for Fig 2. For both Entities and attributes (5 marks) in ER And Relationships and attributes (5 marks) in ER	COLLEGE (<u>CName</u>) STUDENT(<u>SName</u>) ATTENDANCE(<u>StartDate</u>, EndDate) TRANSCRIPT(<u>CourseName</u>, <u>Semester</u>, <u>Year</u>, Grade) DEGREE(<u>DName</u>) COLLEGE_ATTENDED(<u>CName</u>, <u>StartDate</u>) DEGREES(<u>DName</u> , <u>StartDate</u> , Month, Year) ATTENDANCE_RECORD(<u>CourseName</u>, <u>Semester</u>, <u>Year</u>, <u>StartDate</u>) PREVIOUS_EDUCATION(<u>SName</u>, <u>StartDate</u>) **Detailed scheme: 6 tables expected. Duck (-1) for each table missing. There are 3 entities and 3 relationship tables. Take -0.5 for incorrect attribute set. Take -1 if the two tables College_Attended and Attendance_Record are not in the 6 tables.
b. creating the 6 tables (6 marks)	<pre> CREATE TABLE College (CName VARCHAR2(15) NOT NULL, PRIMARY KEY(CName)); CREATE TABLE Student (SName VARCHAR2(15) NOT NULL, PRIMARY KEY(CName)); CREATE TABLE Attendance (StartDate DATE NOT NULL, EndDate DATE, PRIMARY KEY(StartDate)); CREATE TABLE TRANSCRIPT (CourseName VARCHAR2(15) NOT NULL, Semester VARCHAR2(15) NOT NULL, Year VARCHAR2(15) NOT NULL, Grade VARCHAR2(3), PRIMARY KEY(CourseName, Semester, Year)); CREATE TABLE Degree (DName VARCHAR2(15) NOT NULL, PRIMARY KEY(DName)); CREATE TABLE COLLEGE_ATTENDED (CName VARCHAR2(15) NOT NULL, StartDate DATE NOT NULL PRIMARY KEY(CName, StartDate)); FOREIGN KEY (CName) REFERENCES College(CName), FOREIGN KEY (StartDate) REFERENCES Attendance(StartDate); CREATE TABLE DEGREES (DName VARCHAR2(15) NOT NULL, StartDate DATE NOT NULL, Month VARCHAR2(15,</pre>

	<pre> Year VARCHAR2(15), PRIMARY KEY(CName, StartDate) FOREIGN KEY (CName) REFERENCES College(CName), FOREIGN KEY (StartDate) REFERENCES Attend- ance(StartDate)); CREATE TABLE ATTENDANCE_RECORD (CourseName VARCHAR2(15) NOT NULL, Semester VARCHAR2(15) NOT NULL, Year VARCHAR2(15) NOT NULL, StartDate DATE NOT NULL, PRIMARY KEY(CourseName, Semester, Year, StartDate)); FOREIGN KEY (CourseName) REFERENCES Tran- script(CourseName), FOREIGN KEY (Semester) REFERENCES Transcript(Semes- ter), FOREIGN KEY (Year) REFERENCES Transcript(Year). FOREIGN KEY (StartDate) REFERENCES Attend- ance(StartDate)); PREVIOUS_EDUCATION(SName, StartDate) CREATE TABLE PREVIOUS_EDUCATION (SName VARCHAR2(15) NOT NULL, StartDate DATE NOT NULL PRIMARY KEY(SName, StartDate)); FOREIGN KEY (CName) REFERENCES Student(SName), FOREIGN KEY (StartDate) REFERENCES Attend- ance(StartDate)); **Detailed scheme: Take (-1) for each of the 6 tables missing up to a maximum of 3 tables. Take -0.5 for any missing constraints in each table. Take -1 if the two tables College_Attended and Attendance_Record are not in the 6 tables. </pre>
c. SQL query (4 marks)	c. Write an SQL query for the English query “Print the student name, the college name, the start date and end date of a student named Pete Adams”. <pre> Select S.SName, C.CName, A.StartDate, A.EndDate From Student S, College C, Attendance A, Previous_Education P, College_Attended D Where S.Sname = P.Sname and P.StartDate = A.StartDate and A. StartDate = D.StartDate and D.CName= C.CName and S.SName = “Pete Adams”; </pre> **Detailed scheme: Take 1 mark for incorrect project list, 1 mark for incorrect “from table list”, 2 marks off for incorrect where conditions split as -0.5 for any join and other conditions missing up to 4. They do not need to rename.

Question 2: (total marks = 20 for 5 + 5 + 5 + 5)

- (i) Consider the universal relation $R = \{\underline{A}, \underline{B}, C, D, E, F\}$ and the set of functional dependencies in this database is:

FD1: {A, B} → {C, D, E, F},
 FD2: {D} → {E, F}.

Given that the key for R is {A,B},

- Is the relation R in 3NF? Explain your answer using functional dependencies.
(2.5 marks)
- Decompose R into 3NF relations specifying the primary keys and functional dependencies that exist in each of the relations and thus, in the database.
(5 marks)

(ii) Given the STUDENT record relation schema:

STUDENT(Studentid, Courseid, Studentname, Course_grade)

Assume that a student may take multiple courses and that percentage grade is entered. The primary key is (Studentid, Courseid). Other dependencies may exist from the meaning of this domain.

- Is the relation STUDENT in 3NF? Explain your answer using functional dependencies.
(2.5 marks)
- Normalize the relation STUDENT into 3NF set of relations if not in 3NF, showing your new normalized database with its functional dependencies.
(5 marks)

Que 2 (20 marks for all of (i) a, b and (ii) a, b)

(i) (a) Is R in 3NF ? (5 marks)	Consider the universal relation R = {<u>A</u>, <u>B</u>, C, D, E, F} and the set of functional dependencies in this database is: FD1: {A, B} → {C, D, E, F}, FD2: {D} → {E, F}. R is not in 3NF because: With {A,B} as the primary key, all other attributes of R should be solely dependent on {A,B} and not transitively dependent on {A, B} if it is in 3NF. From FD2, there is transitive dependency where the non-key attributes E, F are determined not directly by the key but through another non-key attribute D. These do not allow the relation to be in 3NF. **Detailed scheme: 2 for stating NO, that R is not in 3NF. 3 for at least 2 reasons for FDs that cause the table to not be in 3NF split as -1 for any missing or incorrect violating FD.
(b) Decompose R into 3NF. Show FDs. (5 marks)	To decompose R into 3NF relations, we remove the transitively dependent attributes from the relation and copy the attribute they are dependent on new relation defined from their violating FD2. This sequence of actions is equivalent to removing all the right hand side attributes of the FDs with partial dependency or transitive dependency such as FD2 from R to create the relations: R1: {<u>A</u>, <u>B</u>, C, D} with FD1: {<u>A</u>, <u>B</u>} → {C, D}, R2: (D, E, F) with FD2: {D} → {E, F}.

	The final set of relations in 3NF are R1, R2 as specified above. **Detailed scheme: 3 marks for showing correct decomposed relations. 2 marks for the FDs split as -0.5 for any missing relations or FDs.
(ii) (a) Is STUDENT in 3NF? (5 marks)	STUDENT(<u>Studentid</u>, <u>Courseid</u>, Studentname, Course_grade) Assume that a student may take multiple courses and that percentage grade is entered. The primary key is (<u>Studentid</u>, <u>Courseid</u>). Solution: FDs: FD1: {Studentid, Courseid} → {Studentname, Course_grade} FD2: Studentid → Studentname STUDENT Is not in 3NF because it is not in 2NF: With {<u>Studentid</u>, <u>Courseid</u>} as the primary key, all other attributes of STUDENT should be solely dependent on {<u>Studentid</u>, <u>Courseid</u>} and not transitively dependent on {<u>Studentid</u>, <u>Courseid</u>} if it is in 3NF and to be in 3NF, it also needs to be in 2NF. From FD2, there are partial dependencies that violate the fully functional dependency that do not allow it to be in 2NF since non-key attribute Studentname is determined not by the whole key but by only part of the key. It cannot be 3NF unless it is in 2NF. **Detailed scheme: 3 marks for showing correct decomposed relations. 2 marks for the FDs split as -0.5 for any missing relations or FDs.
(b) Normalize STUDENT. Show FDs. (5 marks)	STUDENT(<u>Studentid</u>, <u>Courseid</u>, Course_grade) With FD1: {Studentid, Courseid} → {Course_grade} STUDENT1(<u>Studentid</u>, Studentname) With FD2: Studentid → Studentname The final set of relations in 3NF are STUDENT and STUDENT1 as specified above. **Detailed scheme: 3 marks for showing correct decomposed relations. 2 marks for the FDs split as -0.5 for any missing relations or FDs

Fig 3: An Instance of a COMPANY relational Database

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

<u>Pname</u>	<u>Pnumber</u>	<u>Plocation</u>	<u>Dnum</u>
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	<u>Sex</u>	<u>Bdate</u>	<u>Relationship</u>
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

EMPLOYEE

<u>Fname</u>	<u>Minit</u>	<u>Lname</u>	<u>Ssn</u>	<u>Bdate</u>	<u>Address</u>	<u>Sex</u>	<u>Salary</u>	<u>Super_ssn</u>	<u>Dno</u>
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

<u>Dname</u>	<u>Dnumber</u>	<u>Mgr_ssn</u>	<u>Mgr_start_date</u>
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Question 3: (total marks = 20 for 10 + 10)

Using the COMPANY database of Fig 3, answer the following questions

by expressing the queries in:

- (i) SQL,
- (ii) Relational algebra,
- (iii) Relational Calculus as well as showing the,
- (iv) result of each query. Note: specify any needed aggregate function or group by attributes within your relational calculus and algebra queries appropriately.

(a) Retrieve the names of male employees who work on project with id 2 and who also have a male dependent. (10 marks)

(b) For each project, list the project name and the minimum hours per week (by all employees) spent on that project. For example, the minimum hours on project 30 (Newbenefits) is 5. (10 marks)

** Some symbols that may be useful for copying are: σ , π , $\leftarrow$, $\bowtie$, $\Join$

Que 3 (10 marks for each of a and b)

(a) Query in SQL, Relational Algebra, Relational Calculus, Result (10 marks)	(a) Retrieve the names of male employees who work on project with id 2 and who also have a male dependent. <u>SQL:</u> <u>SQL:</u> Select e.Lname, e.Fname From Employee e, Works_on w, Dependent d Where e.Ssn = w.Essn and e.Ssn = d.Essn And e.sex = 'M' and w.Pno = 2 and d.sex = 'M'; <u>Relation Algebra:</u> $Emp_M_Dep \leftarrow \sigma_{Employee.sex='M' \text{ and } Dependent.sex='M'} (DEPENDENT \bowtie_{Essn=Ssn} EMPLOYEE))$ /* male emp with male dependents $Emp_Work_P2 \leftarrow \sigma_{Pno=2} (WORKS_ON \bowtie_{Essn=Ssn} (Emp_M_Dep))$ /* male emp who worked on project 2 $Result \leftarrow \pi_{Lname, Fname} (Emp_Work_P2)$ /* names of those male emps who worked on P2 OR A single algebra expression can also be: $\pi_{Lname, Fname} (\sigma_{Employee.sex='M' \text{ and } Dependent.sex='M' \text{ and } Pno=2} (DEPENDENT \bowtie_{Essn=Ssn} EMPLOYEE \bowtie_{Ssn=Essn} WORK_ON))$ <u>Relational Calculus:</u> $\{e.Lname, e.Fname \mid Employee(e) \text{ and } (\exists d) (DEPENDENT(d) \text{ AND } (\exists w) (WORKS_ON(w) \text{ and } e.Sex = 'M' \text{ and } d.sex = 'M' \text{ and } d.Essn=e.Ssn \text{ and } w.Essn = e.Ssn \text{ and } w.Pno = 2))\}$ <u>Query Result:</u> LNAME FNAME ----- Wong Franklin
---	---

	Smith John **Detailed scheme: 2.5 marks for SQL, 2.5 marks for relational algebra, 2.5 marks for relational calculus, 2.5 marks for query result. For each query, duck marks for incorrect project list (-0.5), "From table list"(-0.5), "where clause conditions (-1.5).														
(b) Query in SQL, Relational Algebra, Relational Calculus, Result (10 marks)	(b) For each project, list the project name and the minimum hours per week (by all employees) spent on that project. Select p.Pname, min(w.Hours) From PROJECT p, WORKS_ON w Where p.Pnumber = w.Pno Group by p.Pname; <u>Relational Algebra:</u> $\text{PROJ_HOURS (Pno, Min_Hrs)} \leftarrow \text{Pno} \bowtie_{\text{min Hours}} (\text{WORKS_ON})$ $\text{RESULT} \leftarrow \pi_{\text{Pname, Min_hrs}} (\text{PROJ_HOURS} \bowtie_{\text{Pno = Pnumber}} (\text{PROJECT}))$ OR Single algebra expression: $\pi_{\text{Pname, Min_hrs}} (\text{Pno} \bowtie_{\text{min Hours}} (\text{WORKS_ON} \bowtie_{\text{Pno = Pnumber}} \text{PROJECT}))$ <u>Relational Calculus:</u> $\{w.Pno, \min(w.Hours) \mid \text{WORKS_ON}(w) \text{ AND } ((\exists p)(\text{PROJECT}(p) \text{ AND } p.Pnumber = w.Pno \text{ AND group by } w.Pno))\}$ <u>Result:</u> <table> <thead> <tr> <th>PNAME</th><th>MIN(W.HOURS)</th></tr> </thead> <tbody> <tr><td>Nbenefits</td><td>5</td></tr> <tr><td>Computerize</td><td>10</td></tr> <tr><td>ProductZ</td><td>10</td></tr> <tr><td>Reorganize</td><td>10</td></tr> <tr><td>ProductY</td><td>7.5</td></tr> <tr><td>ProductX</td><td>20</td></tr> </tbody> </table> **Detailed scheme: 2.5 marks **Detailed scheme: 2.5 marks for SQL, 2.5 marks for relational algebra, 2.5 marks for relational calculus, 2.5 marks for query result. For each query, duck marks for incorrect project list (-0.5), "From table list"(-0.5), "where clause conditions (-1.5).	PNAME	MIN(W.HOURS)	Nbenefits	5	Computerize	10	ProductZ	10	Reorganize	10	ProductY	7.5	ProductX	20
PNAME	MIN(W.HOURS)														
Nbenefits	5														
Computerize	10														
ProductZ	10														
Reorganize	10														
ProductY	7.5														
ProductX	20														

Question 4 : (total marks = 10 for 5 + 5)

(a). Given the following B-tree of order p=3 which has already inserted into it the values 40, 17, 32, 20, you are required to insert into it, the sequence of ids 31, 99, 43 and show:

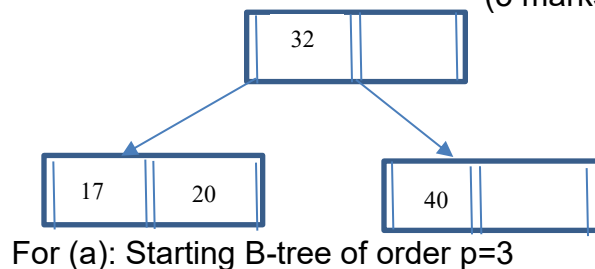
(i) the steps for inserting the given sequence of unique ids for some database records of a

relation, and

(ii) the final B-tree index structure of order $p=3$ (or with maximum of 3 pointers) after the insertions. B-tree is used for speeding up retrieval of records from their primary data file stored on disk.

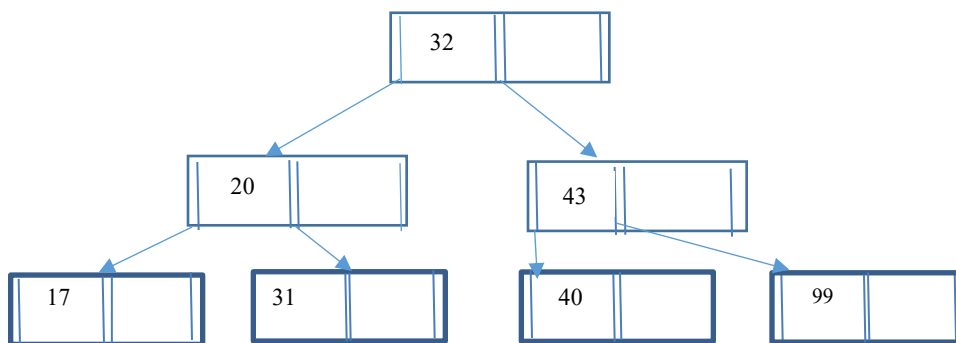
Show the steps through arriving at your final tree before giving your final tree.

(5 marks)



****Detailed scheme: It is 1.5 for correct B-tree after insertion of each of the three numbers for a total of 5 for both 4a(i) and 4a(ii) where 0.5 is for attempt. No attempt is -0.5. Note that inserting correctly into a wrong intermediate tree that will never arrive at the final correct tree does not account for a correct step of insertion.**

Step 3: insert 43 into the above B-tree from step 2 to get:



This is the final B-tree after inserting 31, 99, 43 into B-tree with 40, 17, 32, 20.

Steps for Getting the final B-tree are given below:

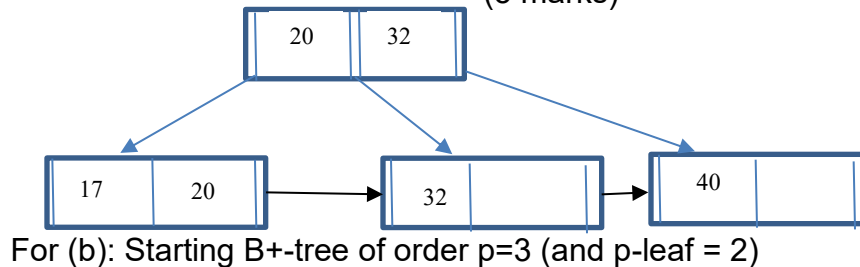
Show this on your own as I did in my classes for each step of building and adding each key to the B-tree

(b). Given the following B+-tree of order $p=3$ (and p -leaf of 2), which has already inserted into it the values 40, 17, 32, 20, you are required to insert into it, the sequence of ids 31, 99, 43 and show:

- (i) the steps for inserting the given sequence of unique ids for some database records of a relation into the B+-tree, and
- (ii) the final B+-tree index structure of order $p=3$ (or with maximum of 3 pointers) after the insertions. B+-tree is used for speeding up retrieval of records from their primary data file stored on disk.

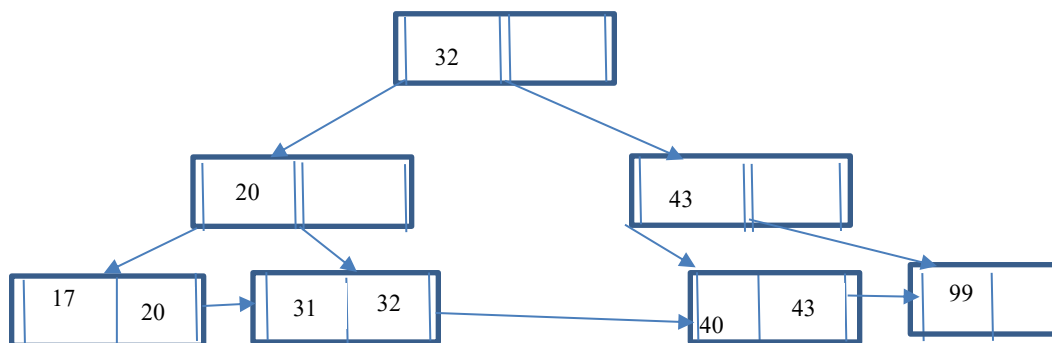
Show the steps through arriving at your final tree before giving your final tree.

(5 marks)



****Detailed scheme:** It is 1.5 for correct B+-tree after insertion of each of the three numbers for a total of 5 for both 4a(i) and 4a(ii) where 0.5 is for attempt. No attempt is -0.5. Note that inserting correctly into a wrong intermediate tree that will never arrive at the final correct tree does not account for a correct step of insertion.

Step 3: The following shows the B+-tree after insertion of 43.



This is the final B+-tree after inserting 31, 99, 43 into the B+-tree with already 40, 17, 32, 20 inserted.

Steps for Getting the final B+-tree are given below:

Show this on your own as I did in my classes for each step of building and adding each key to the B+-tree.